

```
#include <stdio.h>
```

```
int main() {  
    int n, m, i, j;  
  
    printf("Enter number of processes: \n");  
    scanf("%d", &n);  
  
    printf("Enter number of resource types: \n");  
    scanf("%d", &m);  
  
    int alloc[n][m], max[n][m], need[n][m];  
    int avail[m], finish[n], safeSeq[n];  
  
    printf("Enter Allocation Matrix:\n");  
    for(i = 0; i < n; i++) {  
        for(j = 0; j < m; j++) {  
            scanf("%d", &alloc[i][j]);  
        }  
    }  
  
    printf("Enter Max Matrix:\n");  
    for(i = 0; i < n; i++) {  
        for(j = 0; j < m; j++) {  
            scanf("%d", &max[i][j]);  
        }  
    }  
}
```

```
printf("Enter Available Resources: \n");  
for(j = 0; j < m; j++) {  
    scanf("%d", &avail[j]);  
}  
  
for(i = 0; i < n; i++) {  
    for(j = 0; j < m; j++) {  
        need[i][j] = max[i][j] - alloc[i][j];  
    }  
}  
  
for(i = 0; i < n; i++) {  
    finish[i] = 0;  
}  
  
int count = 0;  
  
while(count < n) {  
    int found = 0;  
  
    for(i = 0; i < n; i++) {  
        if(finish[i] == 0) {  
  
            int canExecute = 1;  
  
            for(j = 0; j < m; j++) {  
                if(need[i][j] > avail[j]) {  
                    canExecute = 0;  
                }  
            }  
  
            if(canExecute) {  
                for(j = 0; j < m; j++) {  
                    alloc[i][j] += need[i][j];  
                    need[i][j] = 0;  
                }  
                finish[i] = 1;  
                count++;  
            }  
        }  
    }  
}
```

```

        if(canExecute) {
            for(j = 0; j < m; j++) {
                avail[j] += alloc[i][j];
            }

            safeSeq[count++] = i;
            finish[i] = 1;
            found = 1;
        }
    }

    if(found == 0) {
        break;
    }
}

if(count == n) {
    printf("System is in a SAFE STATE.\n");
    printf("Safe Sequence: ");
    for(i = 0; i < n; i++) {
        printf("%d ", safeSeq[i]);
    }
    printf("\n");
} else {
    printf("System is NOT in a safe state (Deadlock may occur).\n");
}

```

✓ Custom Test

Success ✓

Input:

```

3
3
0 1 0
2 0 0
3 0 3
3 2 2
2 1 1
4 3 3
0 0 0

```

Expected Output :

```

Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).

```

Output:

```

Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).

```

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>
#include <unistd.h>

int main() {
    int shmid;
    char *str;

    // 1. Set size
    int size = 1024;

    // 2. Allocate shared memory
    shmid = shmget((key_t)1234, size, 0666 | IPC_CREAT);

    if (shmid < 0) {
        printf("Shared memory creation failed\n");
        return 1;
    }

    // 3. Attach
    str = (char*) shmat(shmid, NULL, 0);

    // 4. Write data
    sprintf(str, "Hello from Sender Process!");

    printf("Data written to shared memory: %s\n", str);

    // 5. Delay
    sleep(10);

    // 6. Detach
    shmdt(str);

    return 0;
}
```

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>

int main() {
    int shmid;
    char *str;

    // 1. Set size
    int size = 1024;

    // 2. Access shared memory
    shmid = shmget((key_t)1234, size, 0666);

    if (shmid < 0) {
        printf("Shared memory access failed\n");
        return 1;
    }

    // 3. Attach
    str = (char*) shmat(shmid, NULL, 0);

    // 4. Read data
    printf("Data received from shared memory: %s\n", str);

    // 5. Detach
    shmdt(str);

    return 0;
}
```

### Sender Output:

Data written to shared memory: Hello from Sender Process!

### Receiver Output:

Data received from shared memory: Hello from Sender Process!

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

// Semaphores
sem_t empty, full, mutex;

// Producer Thread
void* producer(void* arg) {
    for(int i = 1; i <= 10; i++) {

        // Wait for empty slot
        sem_wait(&empty);

        // Enter critical section
        sem_wait(&mutex);

        // Produce item
        buffer[in] = i;
        printf("Produced: %d\n", i);
        in = (in + 1) % BUFFER_SIZE;

        // Exit critical section
        sem_post(&mutex);

        // Increase full count
        sem_post(&full);

        sleep(1);
    }
    return NULL;
}

int main() {
    pthread_t p, c;

    // 1. Initialize semaphores
    sem_init(&empty, 0, BUFFER_SIZE); // initially all slots empty
    sem_init(&full, 0, 0);             // initially no items
    sem_init(&mutex, 0, 1);            // binary semaphore

    // 2. Create threads
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    // 3. Wait for threads to finish
    pthread_join(p, NULL);
    pthread_join(c, NULL);

    // Destroy semaphores
    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex);

    return 0;
}

```

```
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
Produced: 6
Consumed: 6
Produced: 7
Consumed: 7
Produced: 8
Consumed: 8
Produced: 9
Consumed: 9
Produced: 10
Consumed: 10
```

```

#include <stdio.h>

int main() {
    int m, n;

    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);

    int block[m], remainingSize[m];

    printf("Enter size of each block:\n");
    for(int i = 0; i < m; i++) {
        scanf("%d", &block[i]);
        remainingSize[i] = block[i];    // initialize remaining memory
    }

    printf("Enter number of processes:\n");
    scanf("%d", &n);

    int process[n];

    printf("Enter size of each process:\n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &process[i]);
    }

    // Best Fit Logic with remaining memory
    for(int i = 0; i < n; i++) {
        int best = -1;

        for(int j = 0; j < m; j++) {
            if(remainingSize[j] >= process[i]) {
                if(best == -1 || remainingSize[j] < remainingSize[best]) {
                    best = j;
                }
            }
        }

        if(best != -1) {
            int fragment = remainingSize[best] - process[i];

            printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
                i + 1, process[i], best + 1, block[best], fragment);

            remainingSize[best] -= process[i];    // update remaining memory
        }
        else {
            printf("Process %d of size %d is not allocated\n",
                i + 1, process[i]);
        }
    }

    return 0;
}

```

▼ Custom Test

Success 

Input :

```
5
100 500 200 300 600
4
212 417 112 426
```

Expected Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 4 of size 300 with
Fragment 88
Process 2 of size 417 is allocated to Block 2 of size 500 with
Fragment 83
Process 3 of size 112 is allocated to Block 3 of size 200 with
Fragment 88
Process 4 of size 426 is allocated to Block 5 of size 600 with
Fragment 174
```

Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 4 of size 300 with Fragment 88
Process 2 of size 417 is allocated to Block 2 of size 500 with Fragment 83
Process 3 of size 112 is allocated to Block 3 of size 200 with Fragment 88
Process 4 of size 426 is allocated to Block 5 of size 600 with Fragment 174
```



```
#include <stdio.h>

int main() {
    int m, n;

    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);

    int blockSize[m], remainingSize[m];

    printf("Enter size of each block:\n");
    for(int i = 0; i < m; i++) {
        scanf("%d", &blockSize[i]);
        remainingSize[i] = blockSize[i]; // fixed typo
    }

    printf("Enter number of processes:\n");
    scanf("%d", &n); // fixed syntax

    int processSize[n];

    printf("Enter size of each process:\n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &processSize[i]);
    }

    // First Fit Allocation
    for(int i = 0; i < n; i++) {
        int allocated = 0;

        for(int j = 0; j < m; j++) {
            if(remainingSize[j] >= processSize[i]) {
                int fragment = remainingSize[j] - processSize[i];

                printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
                    i + 1, processSize[i], j + 1, blockSize[j], fragment);

                remainingSize[j] -= processSize[i];
                allocated = 1;
                break;
            }
        }

        if(!allocated) {
            printf("Process %d of size %d is not allocated\n",
                i + 1, processSize[i]);
        }
    }

    return 0;
}
```

Custom Test

Success

Input :

```
5
100 500 200 300 600
4
212 417 112 426
```

Expected Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 2 of size 500 with
Fragment 288
Process 2 of size 417 is allocated to Block 5 of size 600 with
Fragment 183
Process 3 of size 112 is allocated to Block 2 of size 500 with
Fragment 176
Process 4 of size 426 is not allocated
```

Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 2 of size 500 with Fragment 288
Process 2 of size 417 is allocated to Block 5 of size 600 with Fragment 183
Process 3 of size 112 is allocated to Block 2 of size 500 with Fragment 176
Process 4 of size 426 is not allocated
```

```
#include <stdio.h>

int main() {
    int m, n;

    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);

    int block[m], remainingSize[m];

    printf("Enter size of each block:\n");
    for(int i = 0; i < m; i++) {
        scanf("%d", &block[i]);
        remainingSize[i] = block[i];    // initialize remaining memory
    }

    printf("Enter number of processes:\n");
    scanf("%d", &n);

    int process[n];

    printf("Enter size of each process:\n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &process[i]);
    }

    // Best Fit Logic with remaining memory
    for(int i = 0; i < n; i++) {
        int best = -1;

        for(int j = 0; j < m; j++) {
            if(remainingSize[j] >= process[i]) {
                if(best == -1 || remainingSize[j] < remainingSize[best]) {
                    best = j;
                }
            }
        }

        if(best != -1) {
            int fragment = remainingSize[best] - process[i];

            printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
                i + 1, process[i], best + 1, block[best], fragment);

            remainingSize[best] -= process[i];    // update remaining memory
        }
        else {
            printf("Process %d of size %d is not allocated\n",
                i + 1, process[i]);
        }
    }

    return 0;
}
```

Custom Test

Success

Input :

```
5
100 500 200 300 600
4
212 417 112 426
```

Expected Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 4 of size 300 with
Fragment 88
Process 2 of size 417 is allocated to Block 2 of size 500 with
Fragment 83
Process 3 of size 112 is allocated to Block 3 of size 200 with
Fragment 88
Process 4 of size 426 is allocated to Block 5 of size 600 with
Fragment 174
```

Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 4 of size 300 with Fragment 88
Process 2 of size 417 is allocated to Block 2 of size 500 with Fragment 83
Process 3 of size 112 is allocated to Block 3 of size 200 with Fragment 88
Process 4 of size 426 is allocated to Block 5 of size 600 with Fragment 174
```

```
#include <stdio.h>

int main() {
    int m, n;

    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);

    int blockSize[m], remainingSize[m];

    printf("Enter size of each block:\n");
    for(int i = 0; i < m; i++) {
        scanf("%d", &blockSize[i]);
        remainingSize[i] = blockSize[i]; // fixed typo
    }

    printf("Enter number of processes:\n");
    scanf("%d", &n); // fixed syntax

    int processSize[n];

    printf("Enter size of each process:\n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &processSize[i]);
    }

    // First Fit Allocation
    for(int i = 0; i < n; i++) {
        int allocated = 0;

        for(int j = 0; j < m; j++) {
            if(remainingSize[j] >= processSize[i]) {
                int fragment = remainingSize[j] - processSize[i];
                printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
                    i + 1, processSize[i], j + 1, blockSize[j], fragment);

                remainingSize[j] -= processSize[i];
                allocated = 1;
                break;
            }
        }

        if(!allocated) {
            printf("Process %d of size %d is not allocated\n",
                i + 1, processSize[i]);
        }
    }

    return 0;
}
```

Custom Test

Success

Input :

5  
100 500 200 300 600  
4  
212 417 112 426

Expected Output :

Enter number of memory blocks:  
Enter size of each block:  
Enter number of processes:  
Enter size of each process:  
Process 1 of size 212 is allocated to Block 2 of size 500 with Fragment 288  
Process 2 of size 417 is allocated to Block 5 of size 600 with Fragment 183  
Process 3 of size 112 is allocated to Block 2 of size 500 with Fragment 176  
Process 4 of size 426 is not allocated

Output :

Enter number of memory blocks:  
Enter size of each block:  
Enter number of processes:  
Enter size of each process:  
Process 1 of size 212 is allocated to Block 2 of size 500 with Fragment 288  
Process 2 of size 417 is allocated to Block 5 of size 600 with Fragment 183  
Process 3 of size 112 is allocated to Block 2 of size 500 with Fragment 176  
Process 4 of size 426 is not allocated

```
#include <stdio.h>

int main() {
    int n, frames;

    printf("Enter number of pages in reference string: \n");
    scanf("%d", &n);

    int ref[n];

    printf("Enter the reference string: \n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &ref[i]);
    }

    printf("Enter number of frames: ");
    scanf("%d", &frames);

    int frame[frames];
    int i, j;

    for(i = 0; i < frames; i++) {
        frame[i] = -1;
    }

    int pageFaults = 0, pageHits = 0;
    int index = 0;

    for(i = 0; i < n; i++) {
        int found = 0;

        for(j = 0; j < frames; j++) {
            if(frame[j] == ref[i]) {
                found = 1;
                pageHits++;
                break;
            }
        }

        if(!found) {
            frame[index] = ref[i];
            index = (index + 1) % frames;
            pageFaults++;
        }
    }

    printf("\n\nTotal Page Faults: %d\n", pageFaults);
    printf("Total Page Hits: %d\n", pageHits);

    return 0;
}
```

Custom Test

Success

Input :

7  
1 3 0 3 5 6 3  
3

Expected Output :

Enter number of pages in reference string:  
Enter the reference string:  
Enter number of frames:  
  
Total Page Faults: 6  
Total Page Hits: 1

Output :

Enter number of pages in reference string:  
Enter the reference string:  
Enter number of frames:  
  
Total Page Faults: 6  
Total Page Hits: 1

```

#include <stdio.h>

int main() {
    int pages, frames;

    printf("Enter number of pages in reference string: \n");
    scanf("%d", &pages);

    int ref[pages];

    printf("Enter the reference string: \n");
    for(int i = 0; i < pages; i++) {
        scanf("%d", &ref[i]);
    }

    printf("Enter number of frames: \n");
    scanf("%d", &frames);

    int frame[frames], time[frames];
    int faults = 0, hits = 0, counter = 0;

    // Initialize frames
    for(int i = 0; i < frames; i++) {
        frame[i] = -1;
        time[i] = -1;
    }

    for(int i = 0; i < pages; i++) {
        int found = 0;

        // Check Hit
        for(int j = 0; j < frames; j++) {
            if(frame[j] == ref[i]) {
                hits++;
                counter++;
                time[j] = counter;
                found = 1;
                break;
            }
        }

        // Fault
        if(!found) {
            int pos = -1;

            // Empty frame available
            for(int j = 0; j < frames; j++) {
                if(frame[j] == -1) {
                    pos = j;
                    break;
                }
            }

            // If no empty frame → find LRU
            if(pos == -1) {
                pos = 0;
                for(int j = 1; j < frames; j++) {
                    if(time[j] < time[pos]) {
                        pos = j;
                    }
                }
            }

            frame[pos] = ref[i];
            counter++;
            time[pos] = counter;
            faults++;
        }
    }

    printf("\nTotal Page Faults: %d\n", faults);
    printf("Total Page Hits: %d\n", hits);

    return 0;
}

```

▼ Custom Test

Success 

Input :

```
5
1 2 3 4 5
5
```

Expected Output :

```
Enter number of pages in reference string:
Enter the reference string:
Enter number of frames:

Total Page Faults: 5
Total Page Hits: 0
```

Output :

```
Enter number of pages in reference string:
Enter the reference string:
Enter number of frames:

Total Page Faults: 5
Total Page Hits: 0
```